# Neural Network Training

# Deep learning for visual recognition

**Image Classification**





**"dog"**

**Classification**
**categorical labels**

**Pose Estimation**

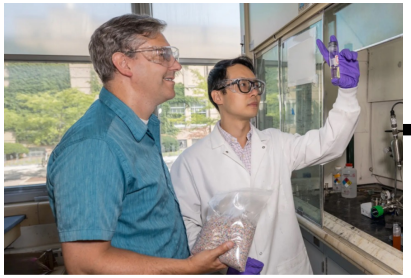






**Regression**
**continuous values**

**VQA / Detection**





**"The person …"**

**Structured prediction**
**list of elements**

# Deep learning for visual recognition

$\{x_i, y_i\}$ $\mathrm{i} = 1 \ldots \mathrm{N}$ training dataset

$x_i$ *i*-th input training example

$y_i$ *i*-th target label

N number of training examples




Goal: predict the target label of unseen inputs.

# Deep learning for visual recognition

- Formulate a visual recognition problem (e.g., image classification) as supervised learning
- Collect a bunch of labeled training data (e.g., ImageNet)
- Design a deep model (w. a loss function) that maps the input to the desired output (e.g., CNN, RNN, or Transformer)
- Train the model using the data by minimizing the training loss (optimization)

# Neural Network Training: The Gory Details

(Or, how to be a helicopter parent to a neural network)



(Or, why AI is a mixed of science and engineering)

Many slides from Lana Lazebnik

# Training as an optimization problem

Neural network: $\hat{y} = f(x; w)$

Training data: $\{\mathrm{x_i},\ \mathrm{y_i}\}$

Loss function: $l(y,\ \hat{y})$

Training: $\mathrm{argmin}_w\ \sum_i l(y_i,\ f(x_i; w))$

# Training as an optimization problem

Training: $\text{argmin}_w \sum_i l(y_i, f(x_i; w))$

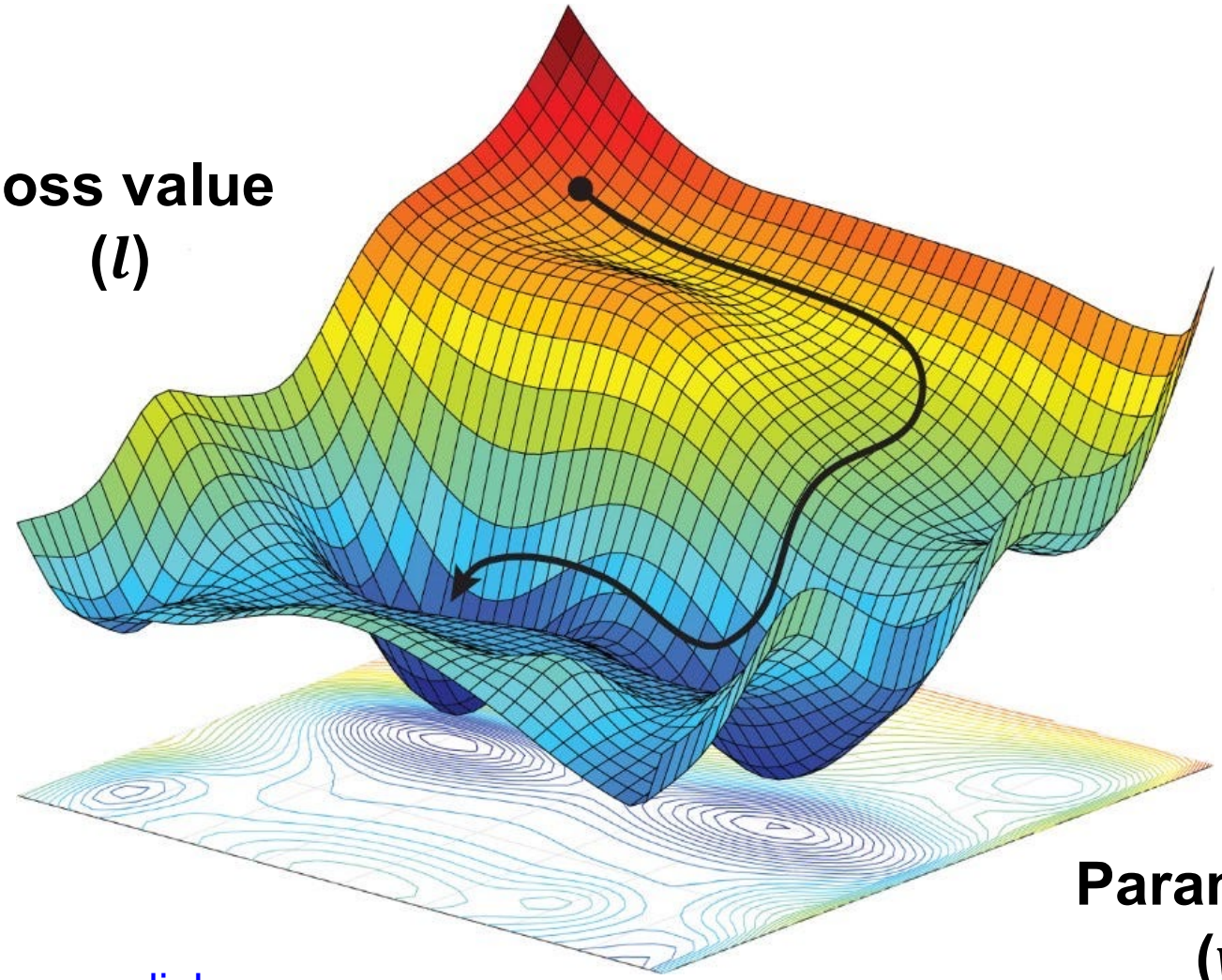



Figure source: link

# Training as an optimization problem

Training: $\text{argmin}_w \sum_i l(y_i, f(x_i; w))$

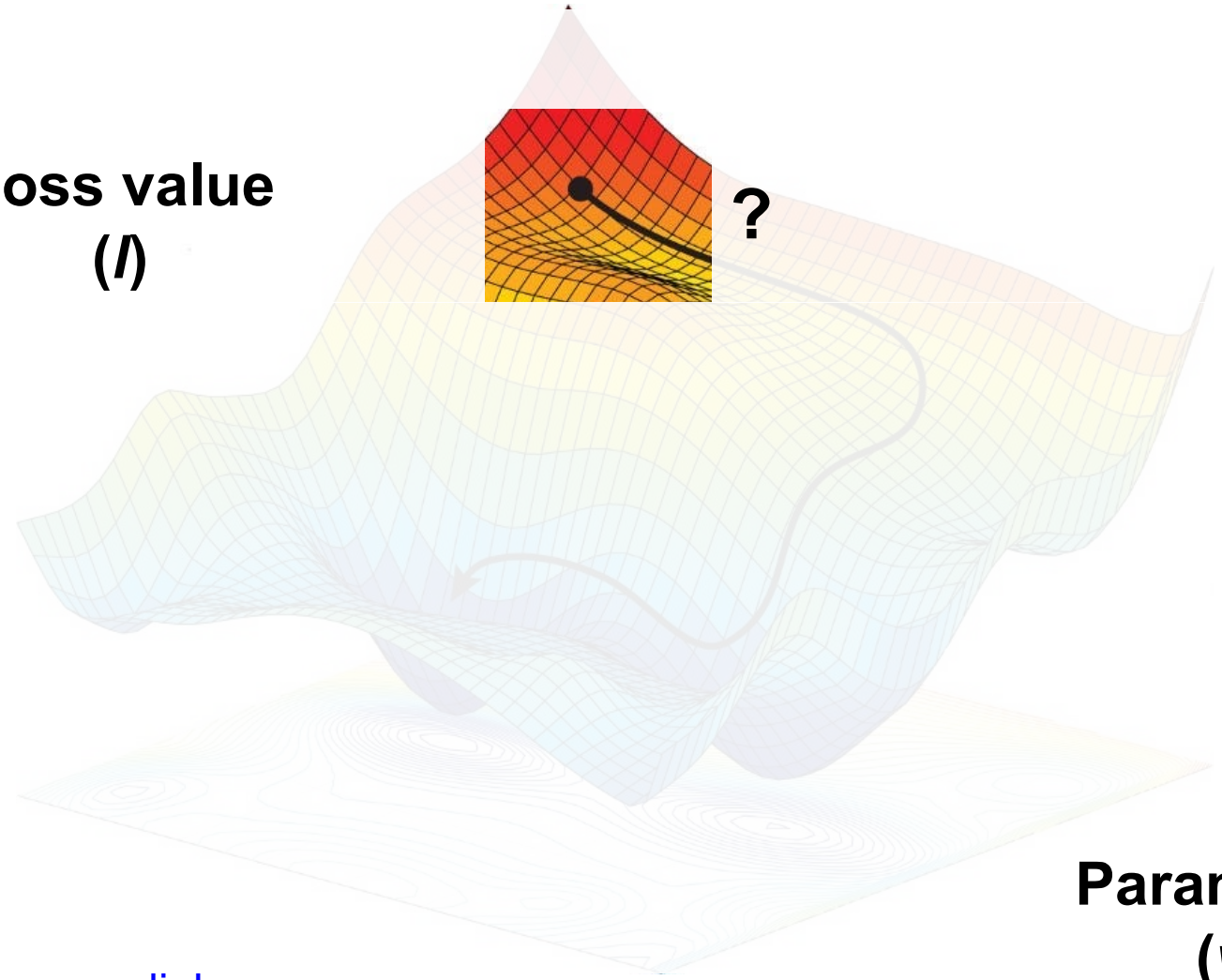



Figure source: link

# An in-depth look into optimization



Caspar David Friedrich,
Wanderer Above a Sea of Fog,
1818

# Optimization with Stochastic Gradient Descent

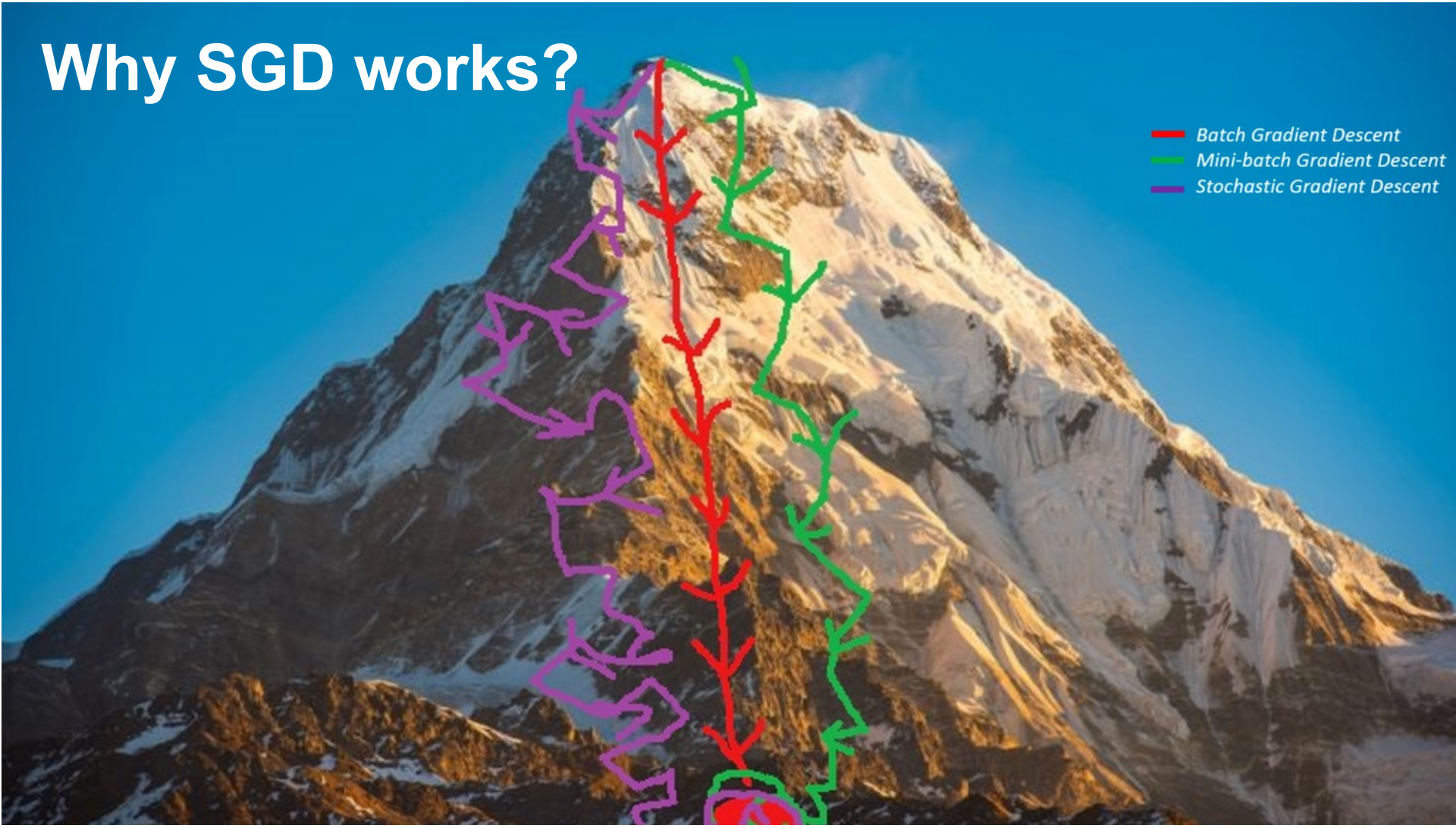



Image Source:
https://towardsdatascience.com/gradient-descent-algorithm-and-its-variants-10f652806a3

# Mini-batch SGD

- Initialize the parameters of the model

- Iterate over epochs
  - Iterate over random mini-batches $(x_1, y_1), \ldots, (x_b, y_b)$
    - Compute gradient of the mini-batch loss:

$$\nabla \hat{L} = \frac{1}{b} \sum_{i=1}^{b} \nabla l(w, x_i, y_i)$$

    - Update parameters with learning rate $\eta$ :

$$w \leftarrow w - \eta \nabla \hat{L}$$

    - Decay the learning rate when necessary
  - Check for convergence, decide whether to decay learning rate

# Mini-batch SGD

- What are the hyperparameters (choices in training)?
  - how to initialize the weight
  - mini-batch size (or how to form a mini-batch)
  - which optimizer to use
  - learning rate
  - learning rate decay schedule
  - deciding when to stop

# Mini-batch SGD

- Initialize the parameters of the model

- Iterate over epochs
  - Iterate over random mini-batches $(x_1, y_1), \ldots, (x_b, y_b)$
    - Compute gradient of the mini-batch loss:

$$\nabla \hat{L} = \frac{1}{b} \sum_{i=1}^{b} \nabla l(w, x_i, y_i)$$

    - Update parameters with learning rate $\eta$ :

$$w \leftarrow w - \eta \nabla \hat{L}$$

    - Decay the learning rate when necessary
  - Check for convergence, decide whether to decay learning rate

# Weight initialization

- What's wrong with initializing all weights to the same number (e.g., zero)?

# Weight initialization

- Typically: initialize to random values sampled from zero-mean Gaussian: $w \sim \mathcal{N}(0, \sigma^2)$
  - Standard deviation matters!
  - Key idea: avoid reducing or amplifying the variance of layer responses, which would lead to vanishing or exploding gradients
- Common initializations
  - $\sigma = 1/\sqrt{n_{\text{in}}}$, where $n_{\text{in}}$ is the number of input dimension
    $\sigma = 2/\sqrt{n_{\text{in}} + n_{\text{out}}}$ (Glorot and Bengio, 2010), where $n_{\text{out}}$ is the number of output dimension
  - $\sigma = \sqrt{2/n_{\text{in}}}$ for ReLU (He et al., 2015)
- Initializing biases: set them to 0

# Weight initialization

Consider a fully connected layer $y = Wx$

$$y_j = \Sigma_{i=1}^{n_{in}} w_{ji} x_i$$

Assume that

- Inputs $x_i$ are independent, with mean 0 and variance $\sigma_x^2$
- Weights $W_{ji}$ are independent, with mean 0 and variance $\sigma_w^2$
- Inputs and weights are independent

$$\mathrm{Var}(y_j) = n_{in} \sigma_x^2 \sigma_w^2$$

To control $\mathrm{Var}(y_j) \sim \sigma_x^2$, we set $\sigma_w = 1/\sqrt{n_{in}}$

See also: http://cs231n.github.io/neural-networks-2/#init

# Mini-batch SGD

- Initialize the parameters of the model

- Iterate over epochs
  - Iterate over random mini-batches $(x_1, y_1), \ldots, (x_b, y_b)$
    - Compute gradient of the mini-batch loss:

$$\nabla \hat{L} = \frac{1}{b} \sum_{i=1}^{b} \nabla l(w, x_i, y_i)$$

    - Update parameters with learning rate $\eta$ :

$$w \leftarrow w - \eta \nabla \hat{L}$$

    - Decay the learning rate when necessary
  - Check for convergence, decide whether to decay learning rate

# Review: AutoDiff & Backpropagation

- Define a neural network as a computational graph
- Run topological sort to determine the order of execution of the operations (input -> output)
- Forward propagation to compute the output
- Compare the output with target using loss function
- Backward propagation to compute the gradients w.r.t. the model parameters
- Use gradient descent to update the parameters

# SGD and mini-batch size

- Larger mini-batches: more expensive and less frequent updates, lower gradient variance, more parallelizable

- In the literature, SGD with larger batches is generally reported to generalize more poorly (e.g., Keskar et al., 2016)
  - But can be made to work by using larger learning rates with larger mini-batches (Goyal et al., 2017)

# Mini-batch SGD

- Initialize the parameters of the model

- Iterate over epochs
  - Iterate over random mini-batches $(x_1, y_1), \ldots, (x_b, y_b)$
    - Compute gradient of the mini-batch loss:

$$\nabla \hat{L} = \frac{1}{b} \sum_{i=1}^{b} \nabla l(w, x_i, y_i)$$

    - Update parameters with learning rate $\eta$ :

$$w \leftarrow w - \eta \nabla \hat{L}$$

    - Decay the learning rate when necessary
  - Check for convergence, decide whether to decay learning rate

# Learning rate decay

- **Exponential decay**: $\eta = \eta_0 e^{-kt}$, where $\eta_0$ and $k$ are hyperparameters, $t$ is the iteration or epoch number
- **$1/t$ decay**: $\eta = \eta_0/(1 + kt)$
- **Step decay**: reduce rate by a constant factor every few epochs, e.g., by 0.5 every 5 epochs, 0.1 every 20 epochs
- **Others:** cosine / poly / …
- **Manual**: watch validation error and reduce learning rate whenever it stops improving

# Diagnosing learning rates

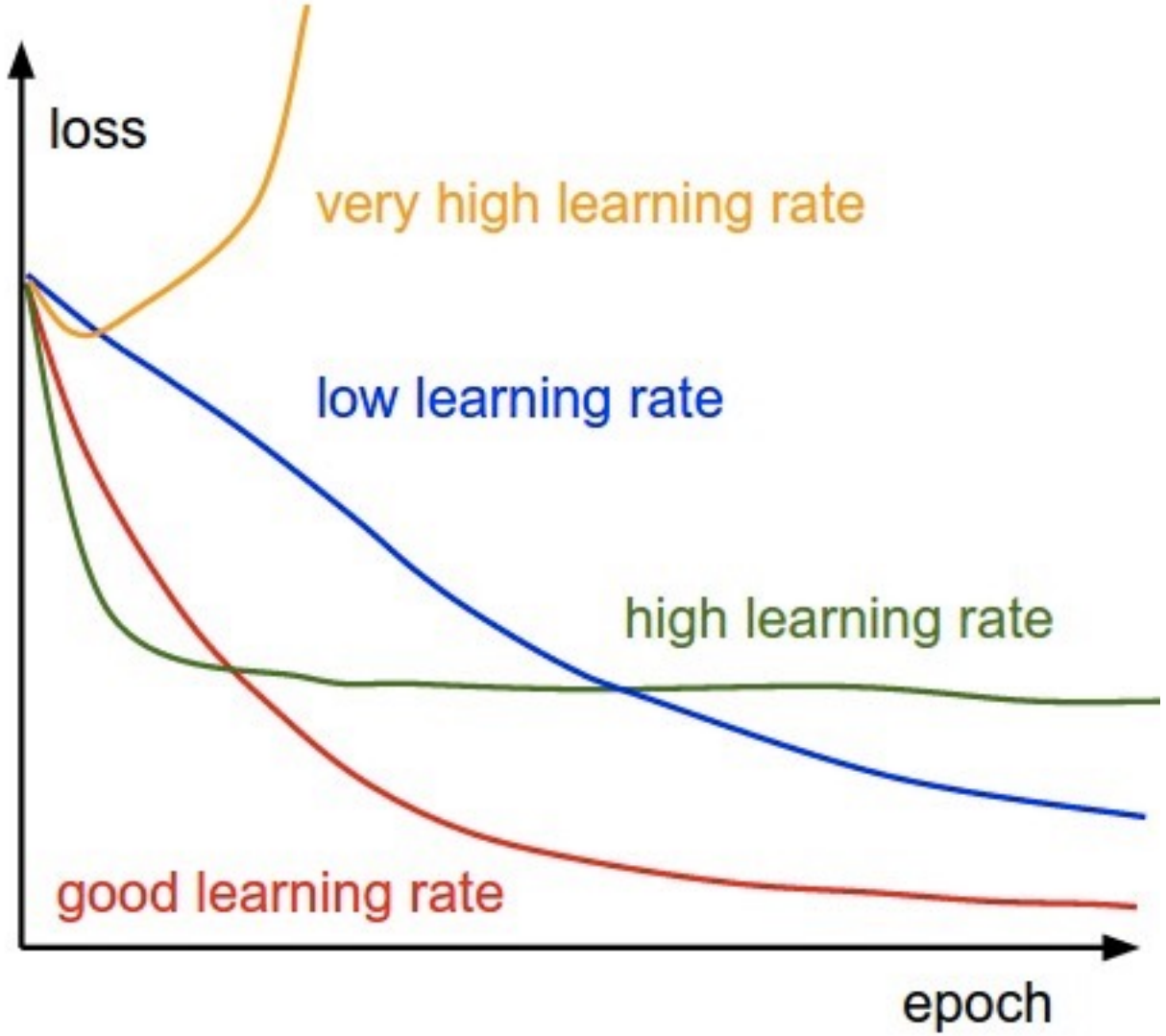
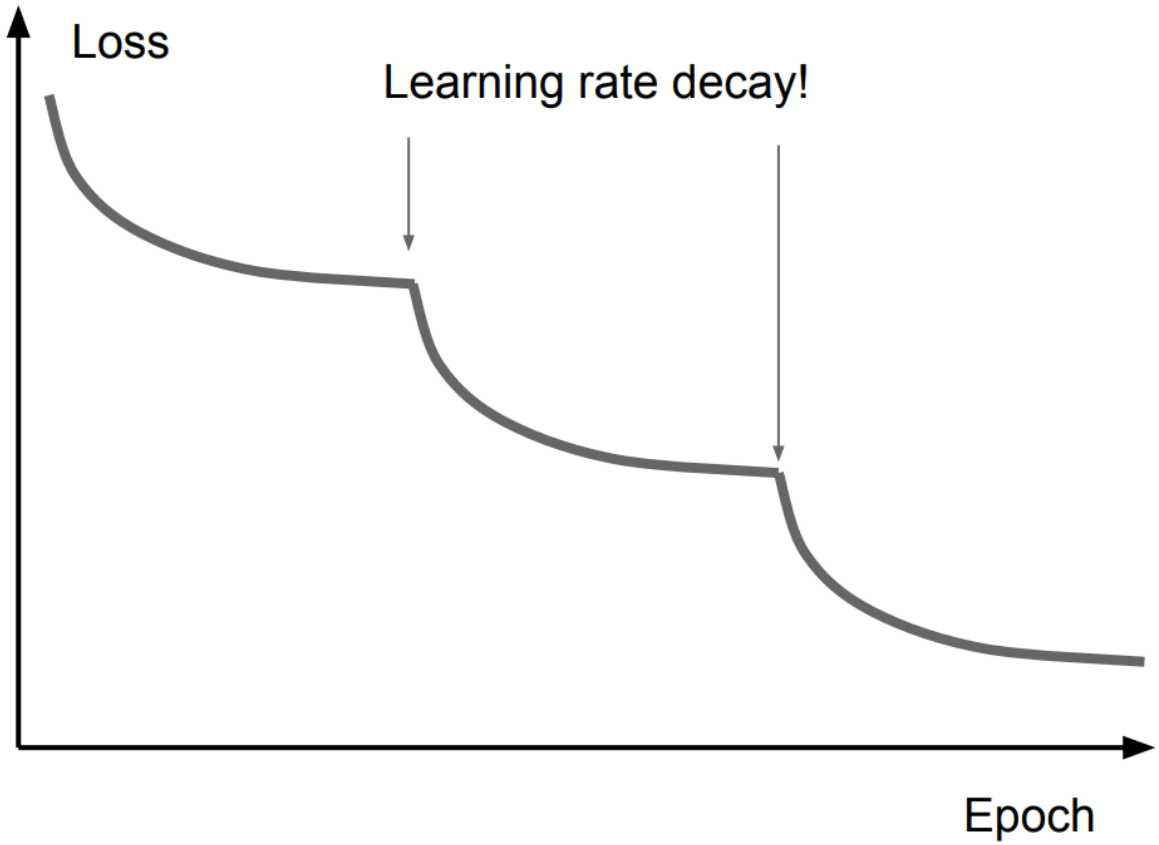



Image source: Stanford CS231n

# A typical phenomenon




- Why does the learning curve look like this?

Image source: Stanford CS231n

# A typical phenomenon

## Possible explanation



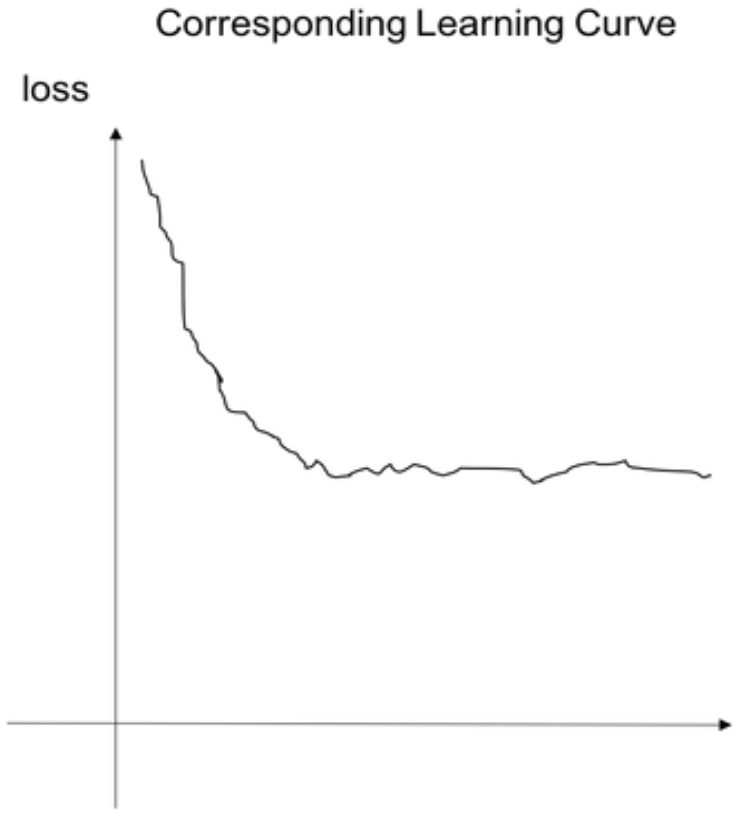
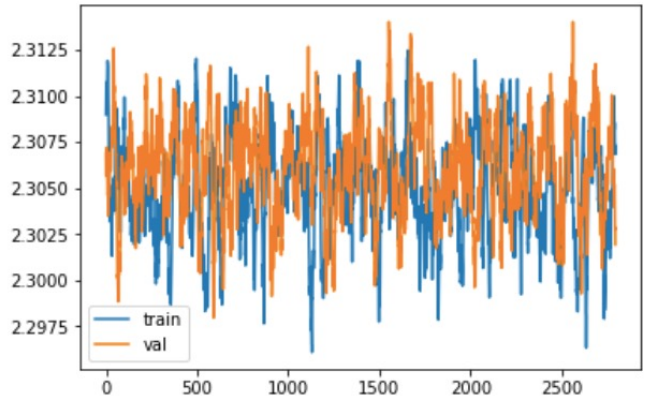
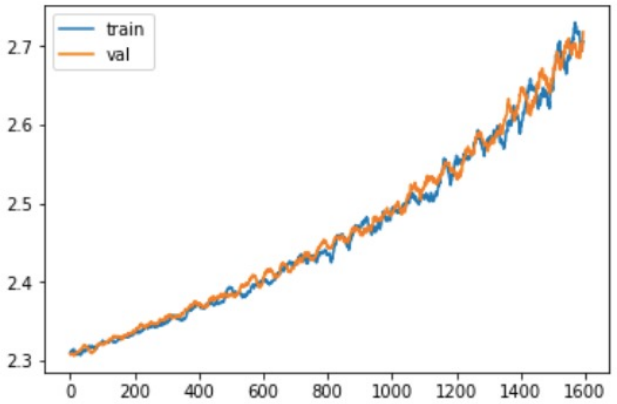




Image source

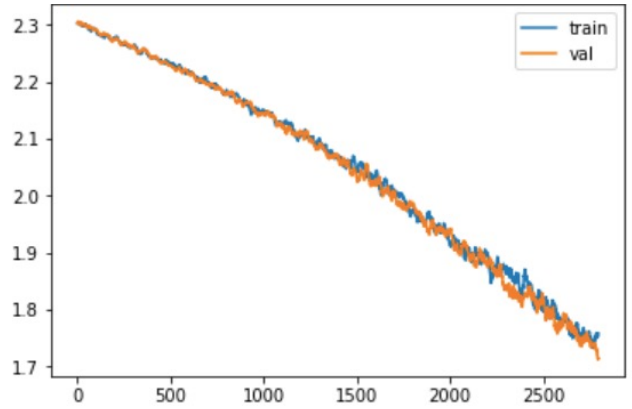
# Debugging learning curves



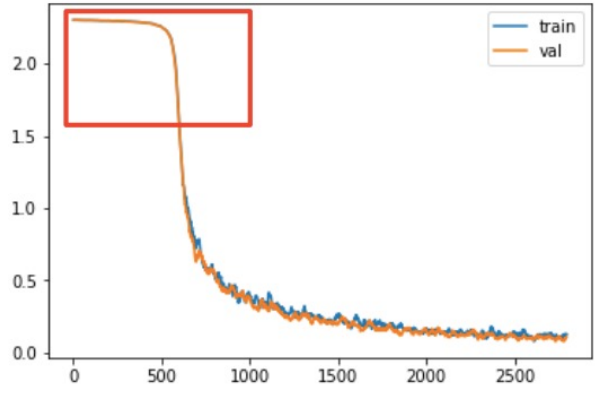

Not training
Bug in update calculation?



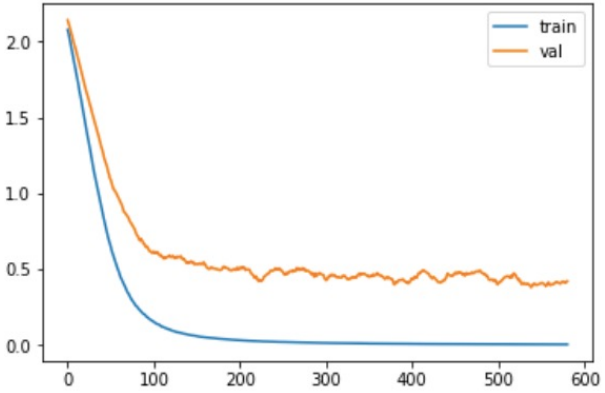

Error increasing
Bug in update calculation?



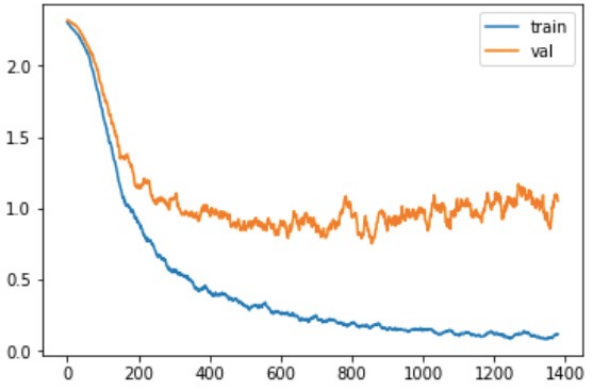

Error decreasing
Not converged yet




Slow start
Suboptimal initialization?




Possible overfitting




Definite overfitting

Image source: [Stanford CS231n]

# Early stopping

- Idea: stop the training when the model starts to overfit
- Monitor validation error to decide when to stop

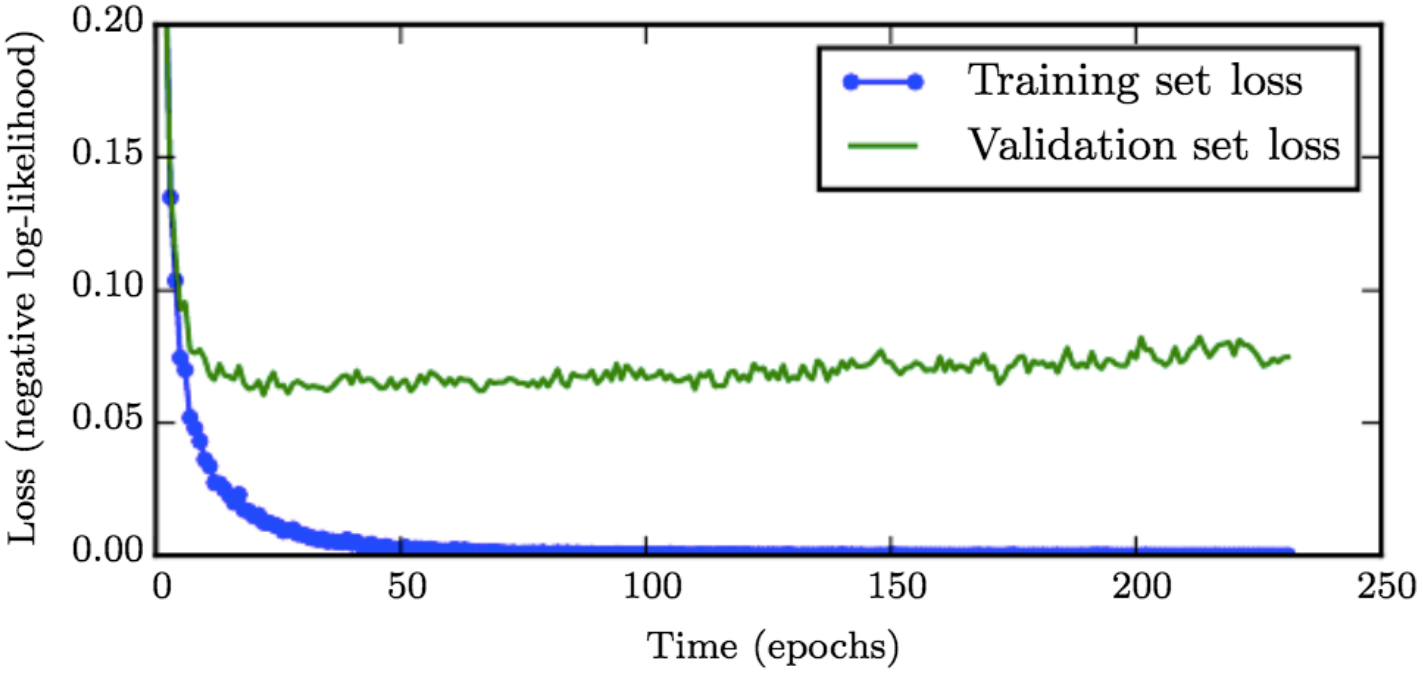



Figure from Deep Learning Book

# Mini-batch SGD with an optimizer

- Initialize the parameters of the model

- Iterate over epochs
  - Iterate over random mini-batches $(x_1, y_1), \ldots, (x_b, y_b)$
    - Compute gradient of the mini-batch loss:

$$\nabla \hat{L} = \frac{1}{b} \sum_{i=1}^{b} \nabla l(w, x_i, y_i)$$

    - Update parameters with learning rate $\eta$ using an optimizer
    - Decay the learning rate when necessary
  - Check for convergence, decide whether to decay learning rate

# Advanced optimizers

- SGD with momentum
- Adagrad
- RSMProp
- Adam

# SGD with momentum




**What will SGD do?**



Key: The expectation of the ”noisy” gradients points to the right direction

Image source

# SGD with momentum

- Introduce a "momentum" variable $m$ and associated "friction" coefficient $\beta$:

$$m \leftarrow \beta m - \eta \nabla L$$

$$w \leftarrow w + m$$

  - When $\beta = 0$, this is vanilla SGD
  - Typical setting: large $\beta = 0.9$

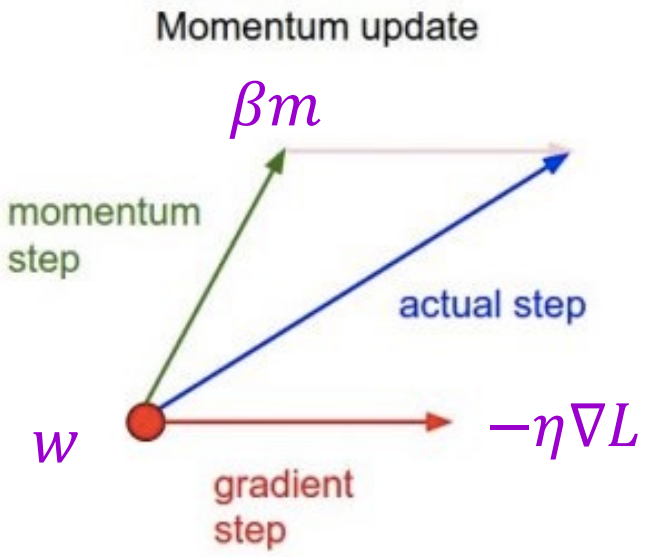



Image source

# SGD with momentum

- Introduce a "momentum" variable $m$ and associated "friction" coefficient $\beta$:

$$m \leftarrow \beta m - \eta \nabla L$$

$$w \leftarrow w + m$$

  - Move faster in directions with consistent gradient
  - Avoid oscillating in directions with large but inconsistent gradients

**Standard SGD**



**SGD with momentum**

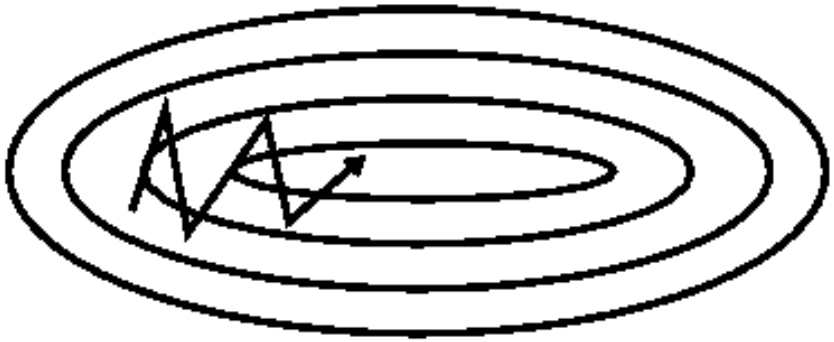


Image source

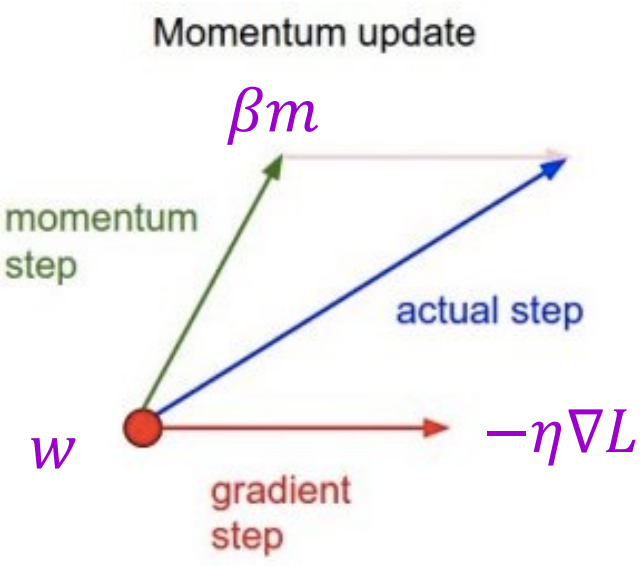
# SGD with momentum

- Introduce a "momentum" variable $m$ and associated "friction" coefficient $\beta$:

$$m \leftarrow \beta m - \eta \nabla L$$

$$w \leftarrow w + m$$

- Nesterov momentum: evaluate gradient at "lookahead" position $w + \beta m$



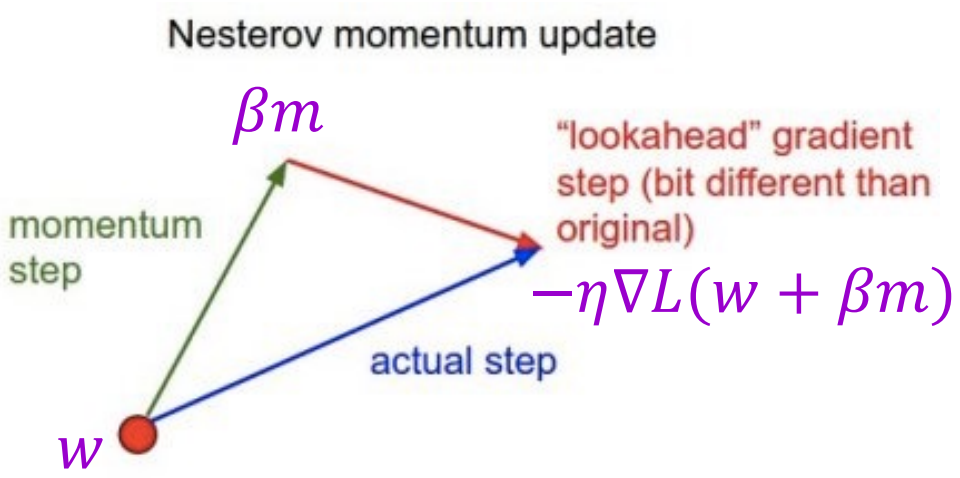




Image source

# Muon: Momentum SGD w. orthogonalization

- Introduce a "momentum" variable $m$ and associated "friction" coefficient $\beta$:

$$m \leftarrow \beta m - \eta \nabla L$$

- Find a better update by orthogonalizing $m$ (assuming $m$ is a matrix ) using

$$\hat{m} \leftarrow \mathrm{argmin}_o ||o - m||_F$$
$$\text{s.t.} \quad o^T o = I \quad \text{or} \quad o o^T = I$$

- The solution is given by SVD approximated by Newton Schulz iterations
- Update the parameter using $\hat{m}$

$$w \leftarrow w + \hat{m}$$

Convergence see: Shen et al. (2025). On the convergence analysis of muon. TMLR

# Adaptive per-parameter learning rates

- Gradients of different layers have different magnitudes

- Is there a smart way to set different learning rates for different parameters?

# Adagrad

- Keep track of history of gradient magnitudes, scale the learning rate for each parameter based on this history:

$$v_k \leftarrow v_k + \left\| \frac{\partial L}{\partial w_k} \right\|^2$$

$$w_k \leftarrow w_k - \frac{\eta}{\sqrt{v_k} + \epsilon} \frac{\partial L}{\partial w_k}$$

  - Parameters with small gradient magnitudes get large updates and vice versa
  - Long-ago gradient magnitudes are not "forgotten" so learning rate decays not too quickly

J. Duchi, Adaptive subgradient methods for online learning and stochastic optimization, JMLR 2011

# RMSProp

- Introduce decay factor $\beta$ (typically $\geq 0.9$) to downweight past history exponentially:

$$v_k \leftarrow \beta v_k + (1-\beta)\left\|\frac{\partial L}{\partial w_k}\right\|^2$$

$$w_k \leftarrow w_k - \frac{\eta}{\sqrt{v_k}+\epsilon}\frac{\partial L}{\partial w_k}$$

http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf

# Adam

- Combine RMSProp with momentum:

$$m \leftarrow \beta_1 m + (1 - \beta_1)\nabla L \qquad \textbf{momentum}$$

$$v_k \leftarrow \beta_2 v_k + (1 - \beta_2)\left\|\frac{\partial L}{\partial w_k}\right\|^2 \qquad \textbf{magnitude}$$

$$w_k \leftarrow w_k - \frac{\eta}{\sqrt{v_k} + \epsilon} m_k \qquad \textbf{gradient}$$

  - Default parameters from paper:
    $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e - 8$
  - Full algorithm includes *bias correction* term to account for $m$ and $v$ starting at 0:

$$\hat{m} = \frac{m}{1-\beta_1^t}, \hat{v} = \frac{v}{1-\beta_2^t}$$ ($t$ is the timestep)

D. Kingma and J. Ba, Adam: A method for stochastic optimization, ICLR 2015

# Summary of advanced optimizers

- SGD with momentum
  (smoothing out the gradient trajectory)
- Adagrad
  (tracking gradient magnitude and recalling the gradients based on the magnitude)
- RSMProp
  (similar to Adagrad, yet better tracking of the gradient magnitude)
- Adam
  (RSMProp + momentum)

# Which optimizer to use in theory?

**Overparameterized Networks:**

- Good local minimum exists in the vicinity of any random initialization
- Vanilla SGD works well even if the model can fit to arbitrary labels [Zhang et al. 2016]

**Infinite-width Networks:**

- Can be approximated by a linear model defined on nonlinear kernels (neural tangent kernel)

[Jacot et al., 2018; Lee et al., 2019; Arora et al., 2019; Li & Liang, 2018; Allen-Zhu et al., 2019; Cao & Gu, 2019; Mei et al., 2019; Du et al., 2019]

# Which optimizer to use in practice?

- Adaptive methods tend to reduce initial training error faster than SGD
  - Adam with default parameters is a popular choice, SGD+momentum may work better but requires more tuning (especially for RNNs)
- However, adaptive methods may quickly plateau on the validation set or do not generalize very well
  - Use Adam first, then switch to SGD?
  - Or just stick with plain old SGD? (Wilson et al., 2017)
- All methods require some level of careful tuning and learning rate control

# Which optimizer to use in practice?

```
model = [Parameter(torch.randn(2, 2, requires_grad=True))]
optimizer = SGD(model, 0.1)
scheduler = ExponentialLR(optimizer, gamma=0.9)

for epoch in range(20):
    for input, target in dataset:
        optimizer.zero_grad()
        output = model(input)
        loss = loss_fn(output, target)
        loss.backward()
        optimizer.step()
    scheduler.step()
```

https://pytorch.org/docs/stable/optim.html

# Recap: Mini-batch SGD

- Initialize the parameters of the model

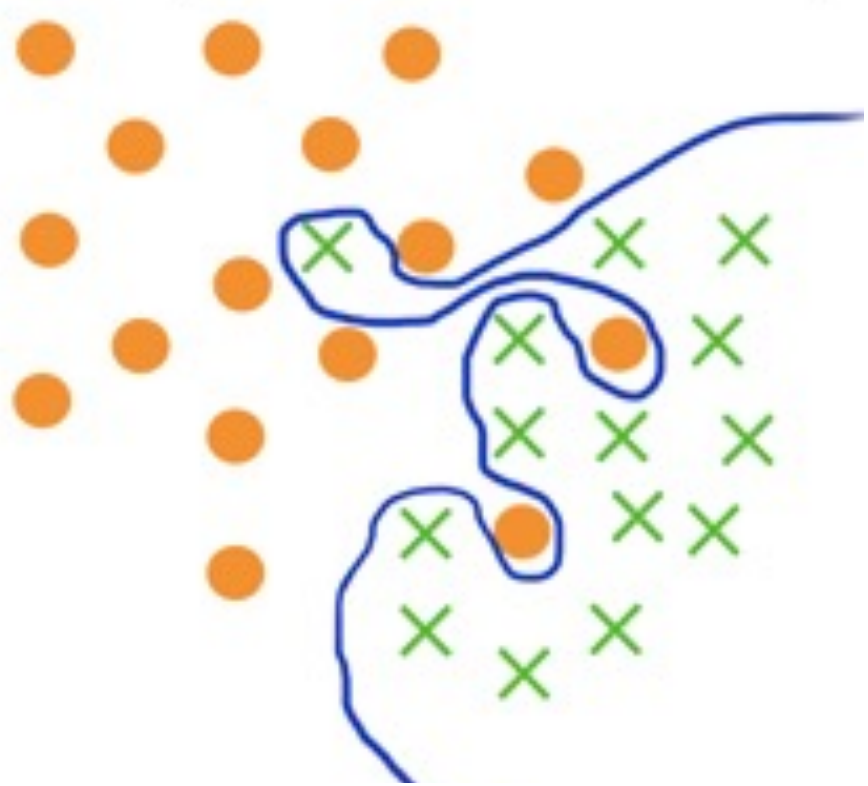
- Iterate over epochs
  - Iterate over random mini-batches $(x_1, y_1), \ldots, (x_b, y_b)$
    - Compute gradient of the mini-batch loss:

$$\nabla \hat{L} = \frac{1}{b} \sum_{i=1}^{b} \nabla l(w, x_i, y_i)$$

    - Update parameters with learning rate $\eta$ using an optimizer
    - Decay the learning rate when necessary
  - Check for convergence, decide whether to decay learning rate

# Regularization

- Techniques for controlling the capacity of a neural network to prevent overfitting



# L2 regularization

- Regularized objective (preference of small weights):

$$\hat{L}(w) = \frac{\lambda}{2}\|w\|_2^2 + \sum_{i=1}^{n} l(w, x_i, y_i)$$

- Gradient of objective:

$$\nabla\hat{L}(w) = \lambda w + \sum_{i=1}^{n} \nabla l(w, x_i, y_i)$$

- SGD update:

$$w \leftarrow w - \eta\big(\lambda w + \nabla l(w, x_i, y_i)\big)$$

$$w \leftarrow (1 - \eta\lambda)w - \eta\nabla l(w, x_i, y_i)$$

- Interpretation: weight decay

# Optimizer with L2 regularization

- SGD w. weight decay
  - Simply replace the gradient

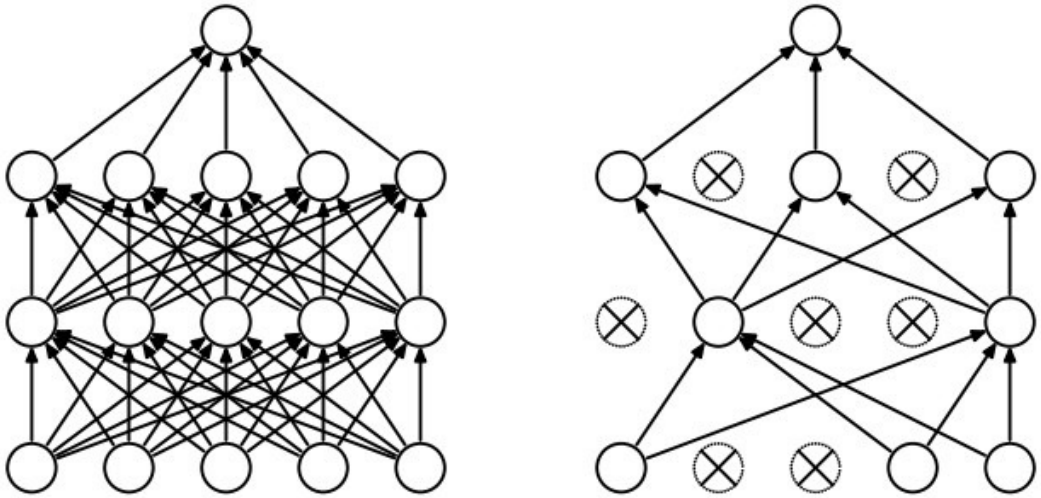
- Adaptive gradient methods w. weight decay?
  - Careful design is needed
  - Example: AdamW

# Other types of regularization

- Adding noise to the inputs
  - Recall motivation of max margin criterion
  - In simple scenario (linear model, quadratic loss, Gaussian noise), this is equivalent to weight decay
  - More general form: data augmentation (later slides) or adversarial training (later lectures)
- Adding noise to the intermediate responses (e.g., Dropout)
- Adding noise to the weights

# Regularization: Dropout

- At training time, in each forward pass, turn off some neurons with probability p
- At test time, to have deterministic behavior, multiply output of neuron by p



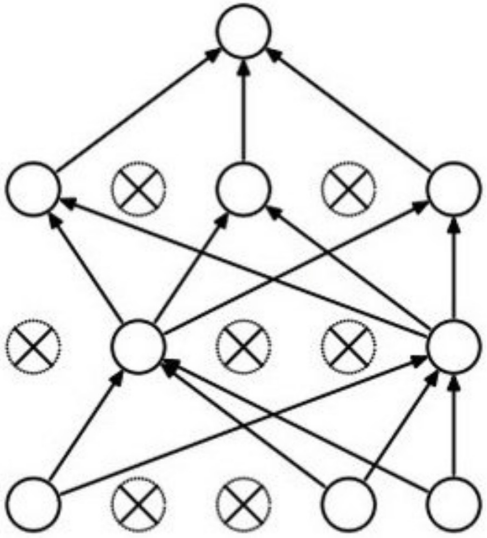
(a) Standard Neural Net

(b) After applying dropout.

N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: A Simple Way to Prevent Neural Networks from Overfitting, JMLR 2014

# Regularization: Dropout

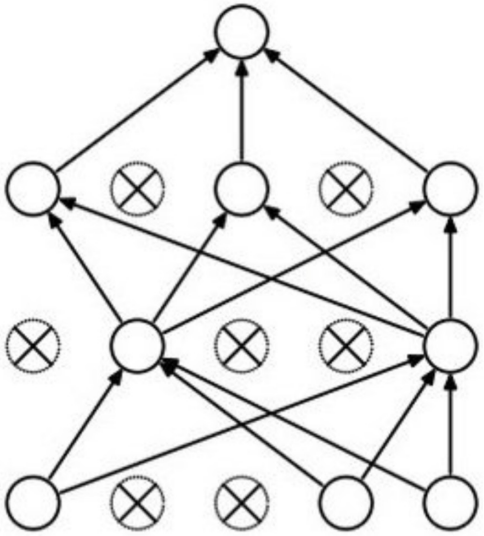
Waaaait a second…
How could this possibly be a good idea?



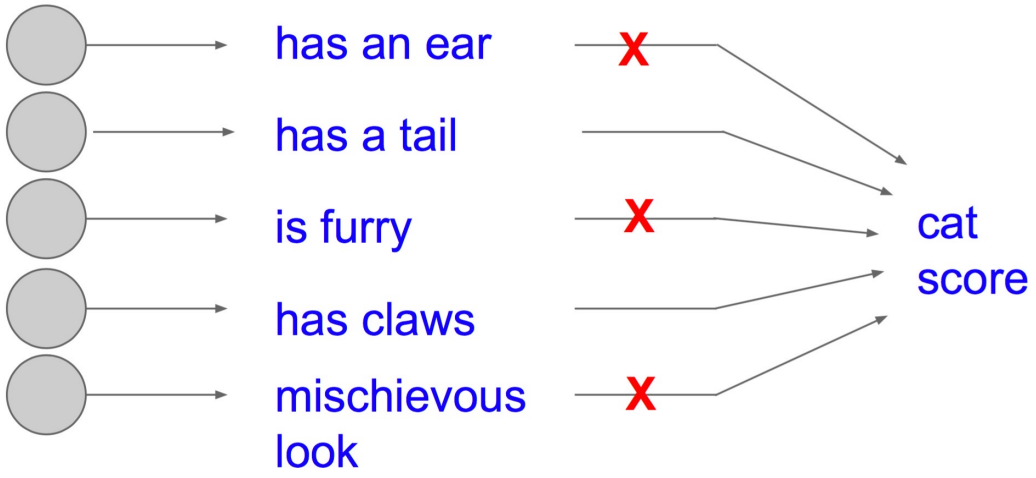
N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: A Simple Way to Prevent Neural Networks from Overfitting, JMLR 2014

# Regularization: Dropout

Waaaait a second…
How could this possibly be a good idea?



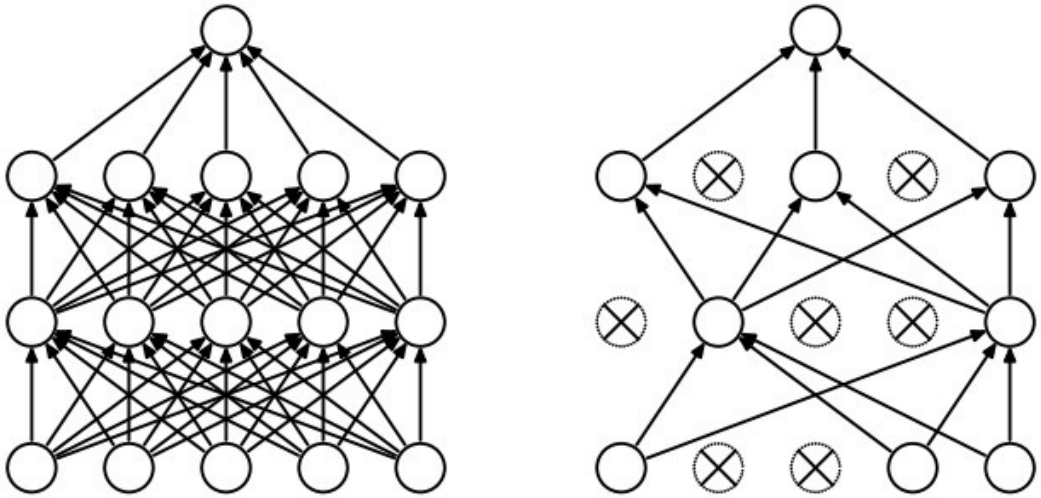
Forces the network to have a redundant representation.




N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: A Simple Way to Prevent Neural Networks from Overfitting, JMLR 2014

# Regularization: Dropout

- Intuitions
  - Prevent “co-adaptation” of units, increase robustness to noise
  - Train *implicit ensemble* (each configuration is one model that gets trained on a single data point)



(a) Standard Neural Net

(b) After applying dropout.

N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: A Simple Way to Prevent Neural Networks from Overfitting, JMLR 2014

# Current status of Dropout

- Against
  - Slows down convergence
  - Made redundant by batch normalization or possibly even clashes with it
  - Unnecessary for larger datasets or with sufficient data augmentation
- In favor
  - Can still help in certain scenarios: e.g., used in vision Transformers

# Bag of Tricks



# Data augmentation

- Introduce transformations not adequately sampled in the training data
  - Geometric: flipping, rotation, shearing, multiple crops
  - Photometric: color transformations
  - Other: add noise, compression artifacts, lens distortions, etc.



Image source

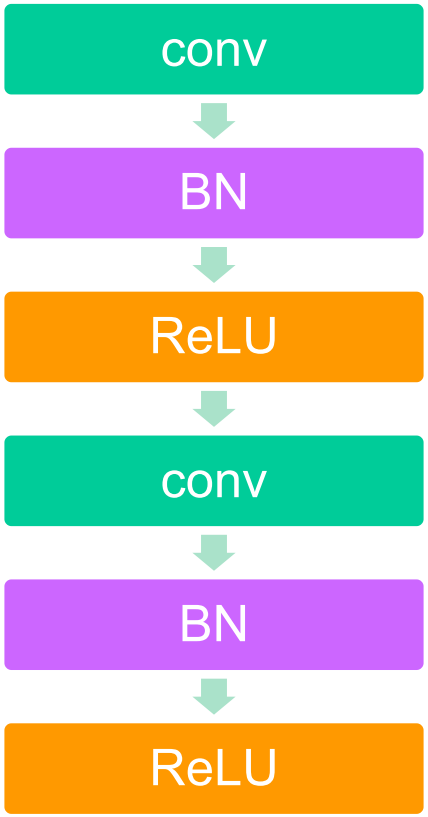
# Data preprocessing

- Zero centering
  - Subtract *mean image* – all input images need to have the same resolution
  - Subtract *per-channel means* – images don't need to have the same resolution
- Optional: rescaling – divide each value by (per-pixel or per-channel) standard deviation

- Be sure to apply the same transformation at training and test time!
  - Save training set statistics and apply to test data

# Review: Batch normalization

- BN: normalize by batch mean and variance
- Insert BN layers right after conv or FC layers, before ReLU nonlinearity
- Make sure they are properly initialized




S. Ioffe, C. Szegedy, Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, ICML 2015

# Batch normalization

- Benefits
  - Prevents exploding and vanishing gradients
  - Keeps most activations away from saturation regions of non-linearities
  - Accelerates convergence of training
  - Makes training more robust w.r.t. hyperparameter choice, initialization
- Pitfalls
  - Behavior depends on composition of minibatches, can lead to hard-to-catch bugs if there is a mismatch between training and test regime (example)

S. Ioffe, C. Szegedy, Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, ICML 2015

# Label smoothing

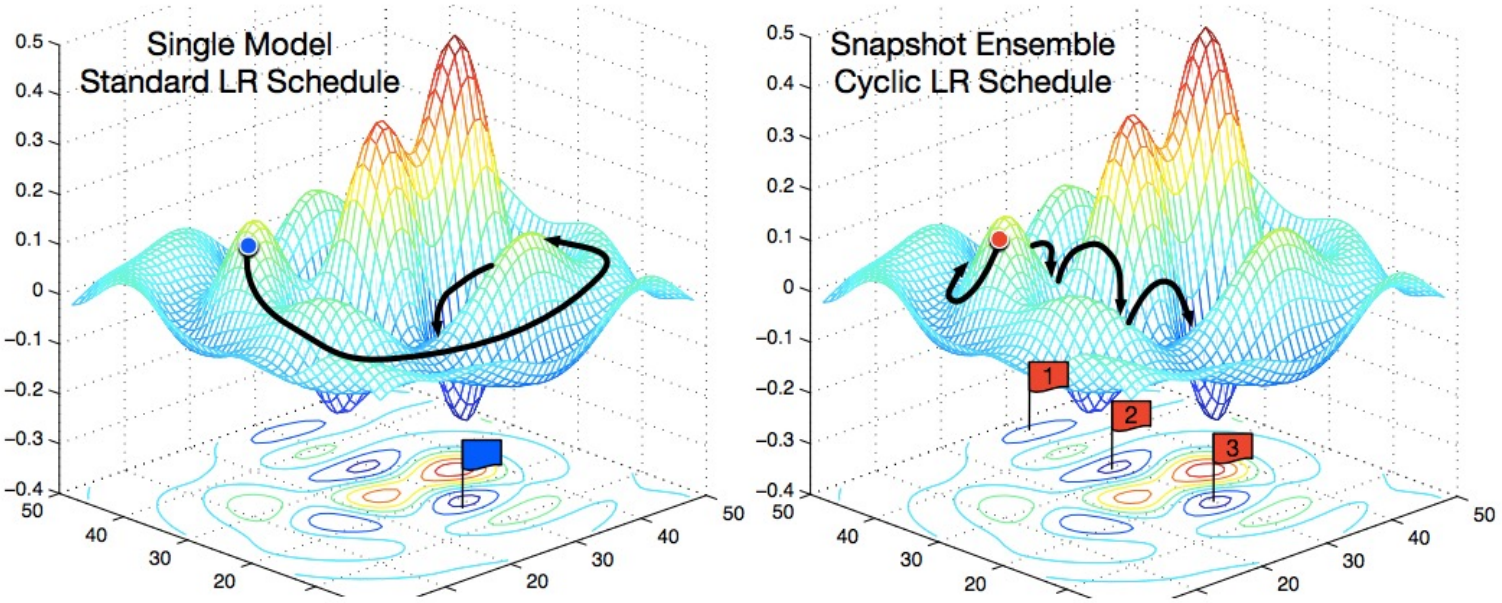
- **Idea:** avoid overly confident predictions, account for *label noise*

- When using softmax loss, replace hard 1 and 0 prediction targets with “soft” targets of $1 - \epsilon$ and $\frac{\epsilon}{C-1}$

- Used in Inception-v2 architecture

# Test time

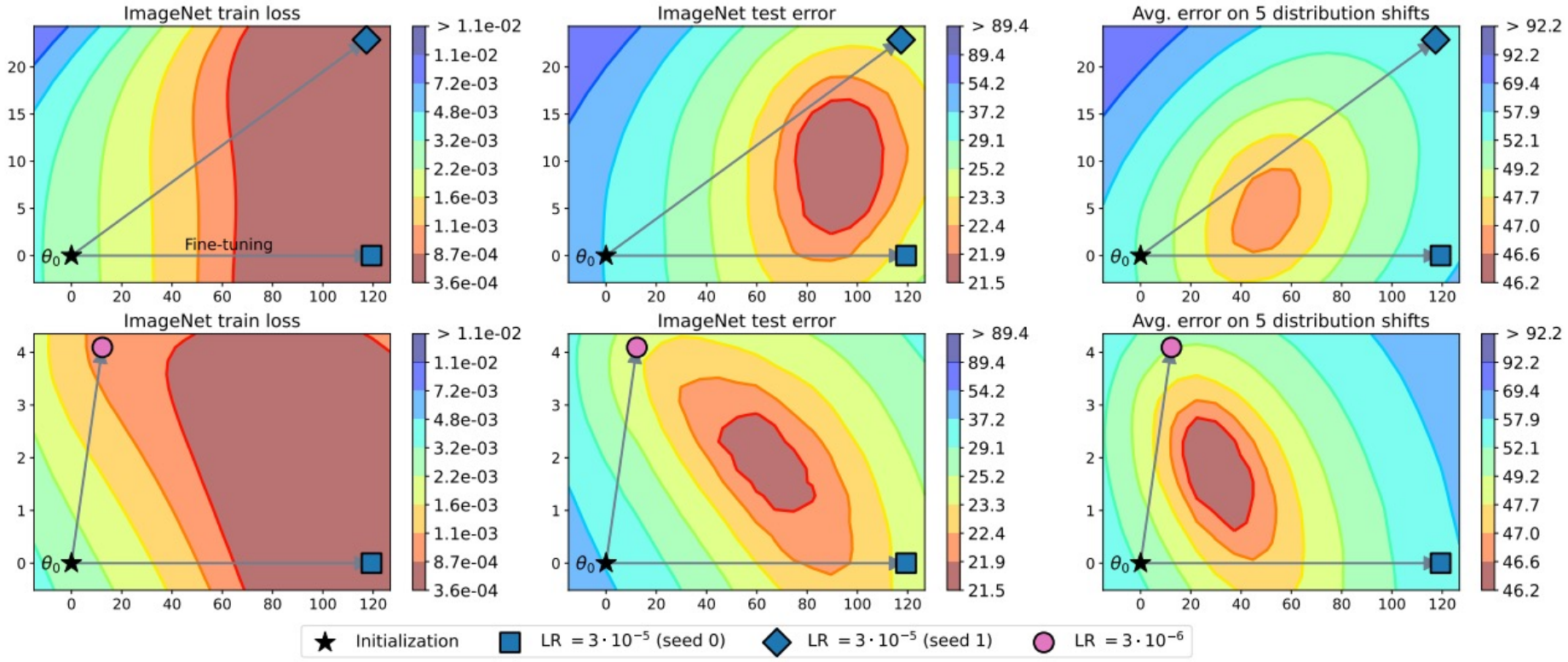
- **Ensembles:** train multiple independent models, then average their predicted label distributions
  - Gives 1-2% improvement in most cases
  - Can take multiple snapshots of models obtained during training, especially if you cycle the learning rate




G. Huang et al., Snapshot ensembles: Train 1, get M for free, ICLR 2017

# Test time

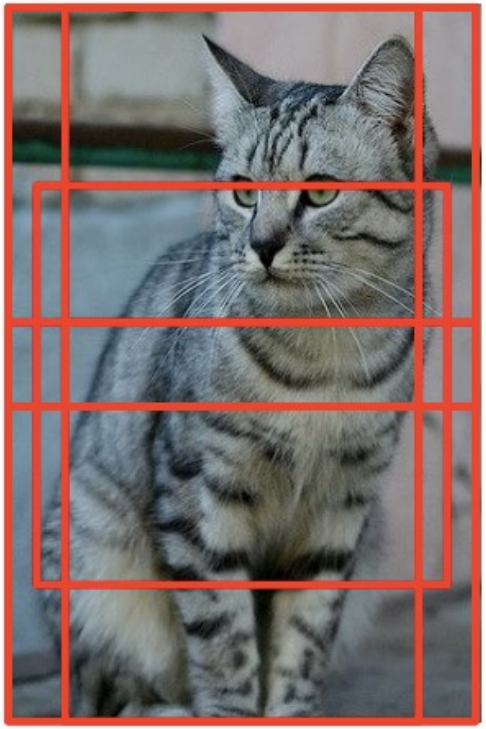
- **Ensembles:** train multiple independent models, then average their weights for prediction




Wortsman et al., Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time, ICML 2022

# Test time

- Average predictions across multiple crops of test image
  - There is a more elegant way to do this with *fully convolutional networks* (FCNs)



# Attempt at a conclusion

- Training neural networks is still more of an art
- Process requires close “babysitting”
- For many techniques, the reasons why, when, and whether they work are in active dispute
- Read everything but don’t trust anything
- It all comes down to (principled) trial and error

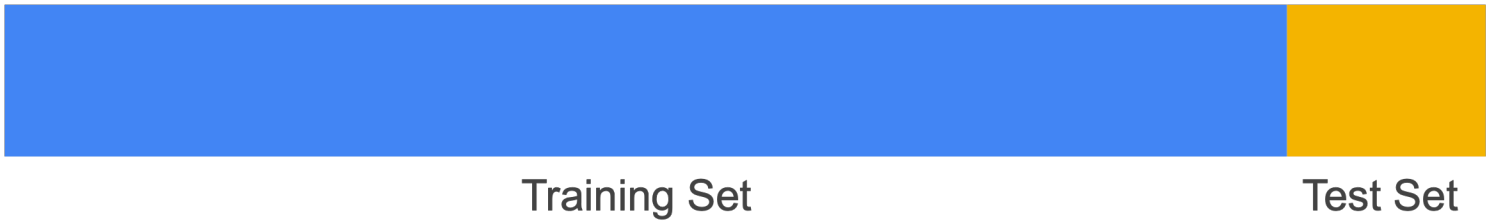


# Additional Slides: Rethinking generalization of deep models

Zhang, C., Bengio, S., Hardt, M., Recht, B., & Vinyals, O. (2021). Understanding deep learning (still) requires rethinking generalization. *CACM*, *64*(3), 107-115.

# ML 101: Bias and Variance

- Use a hold-out test set

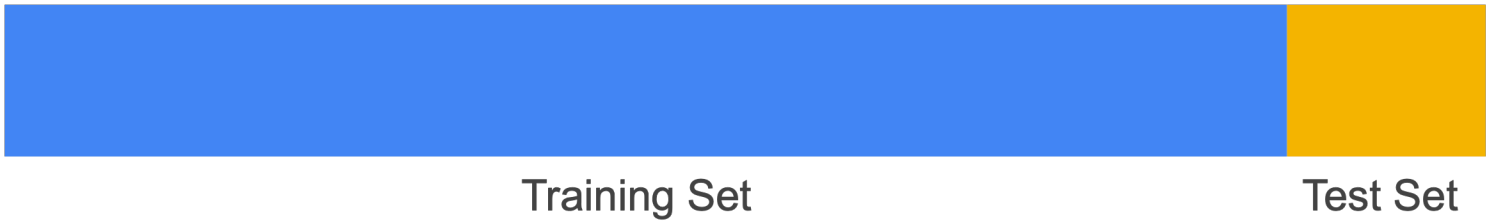



- Data distribution *D*: $x = f(x) + \epsilon$, $\epsilon$ as noise with 0 mean and variance $\sigma$

- Learning a function $\hat{f}(x)$

# ML 101: Bias and Variance

- Use a hold-out test set

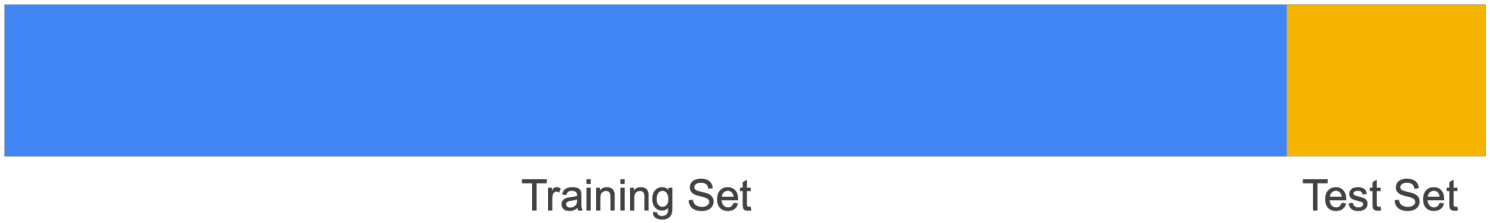



- Average error on D

$$\mathrm{E}_{D,\epsilon}\left[\left(y-\hat{f}(x)\right)^2\right] = Bias^2 + Var + \sigma^2$$

# ML 101: Bias and Variance

- Use a hold-out test set



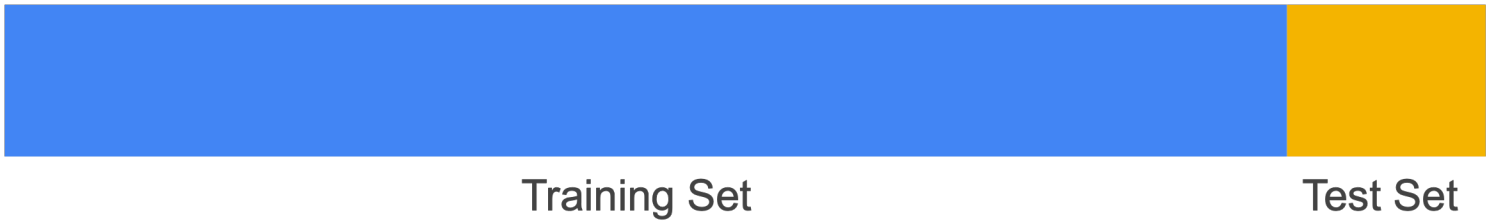

- Average error on D

$$\mathrm{E}_{D,\epsilon}\left[\left(y-\hat{f}(x)\right)^2\right] = \underline{Bias^2} + Var + \sigma^2$$

Average prediction error

# ML 101: Bias and Variance

- Use a hold-out test set



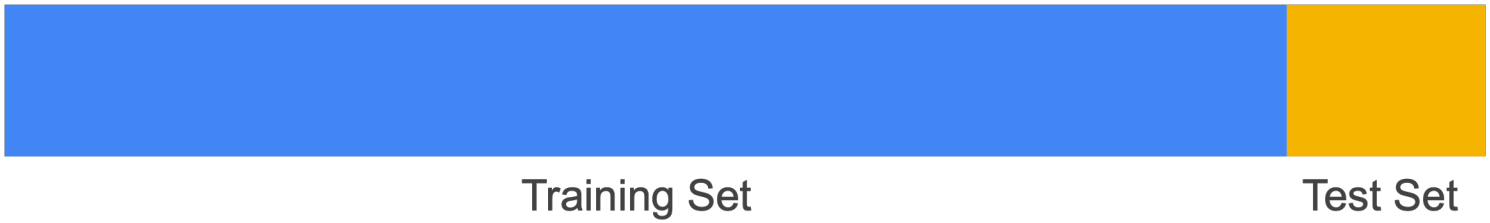

- Average error on D

$$\mathrm{E}_{D,\epsilon}\left[\left(y-\hat{f}(x)\right)^2\right] = Bias^2 + Var + \sigma^2$$

Variance of the prediction

# ML 101: Bias and Variance

- Use a hold-out test set



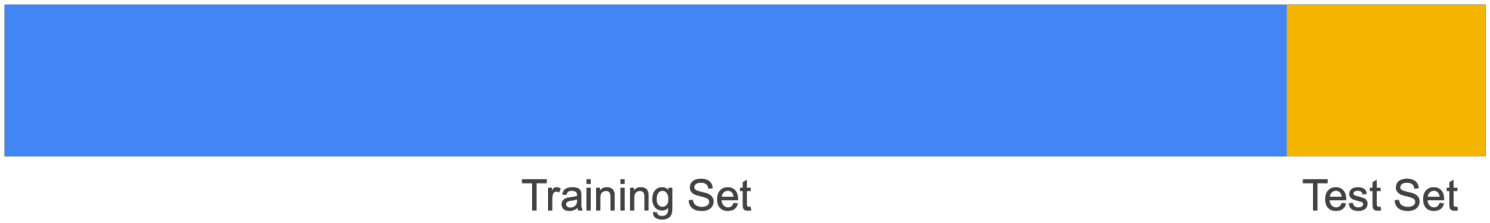

- Average error on D

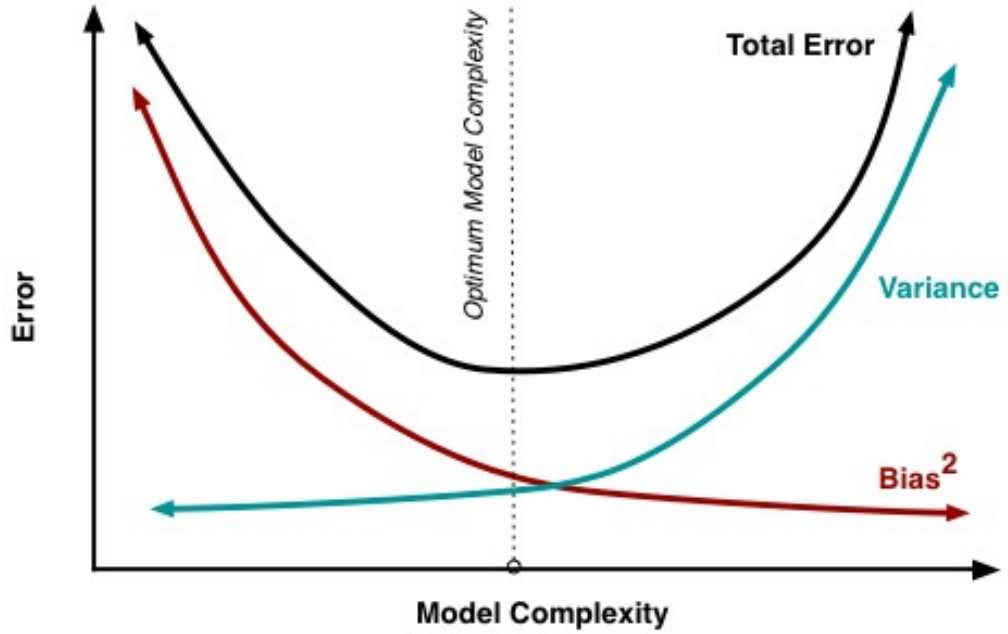
$$\mathrm{E}_{D,\epsilon}\left[\left(y-\hat{f}(x)\right)^2\right] = Bias^2 + Var + \sigma^2$$

Irreducible error

# ML 101: Bias and Variance Tradeoff

- Use a hold-out test set

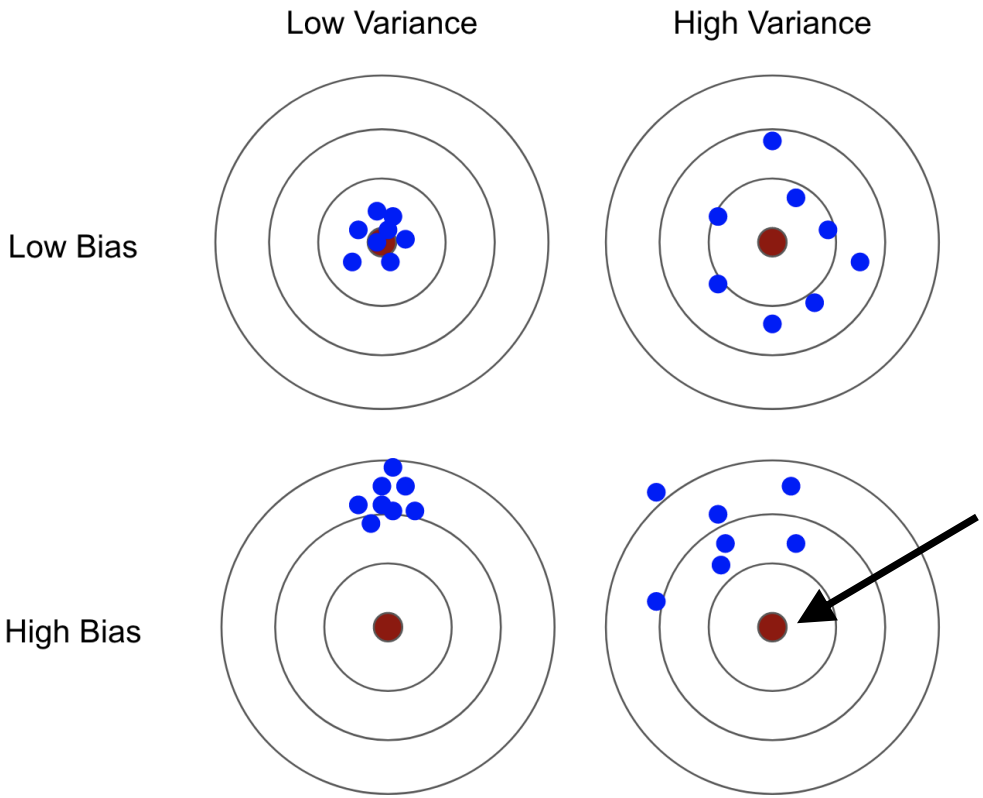


# ML 101: Revisited

- Use a hold-out test set

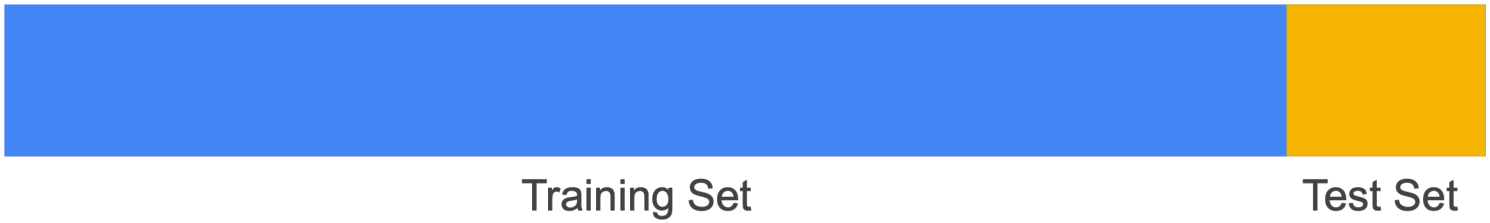


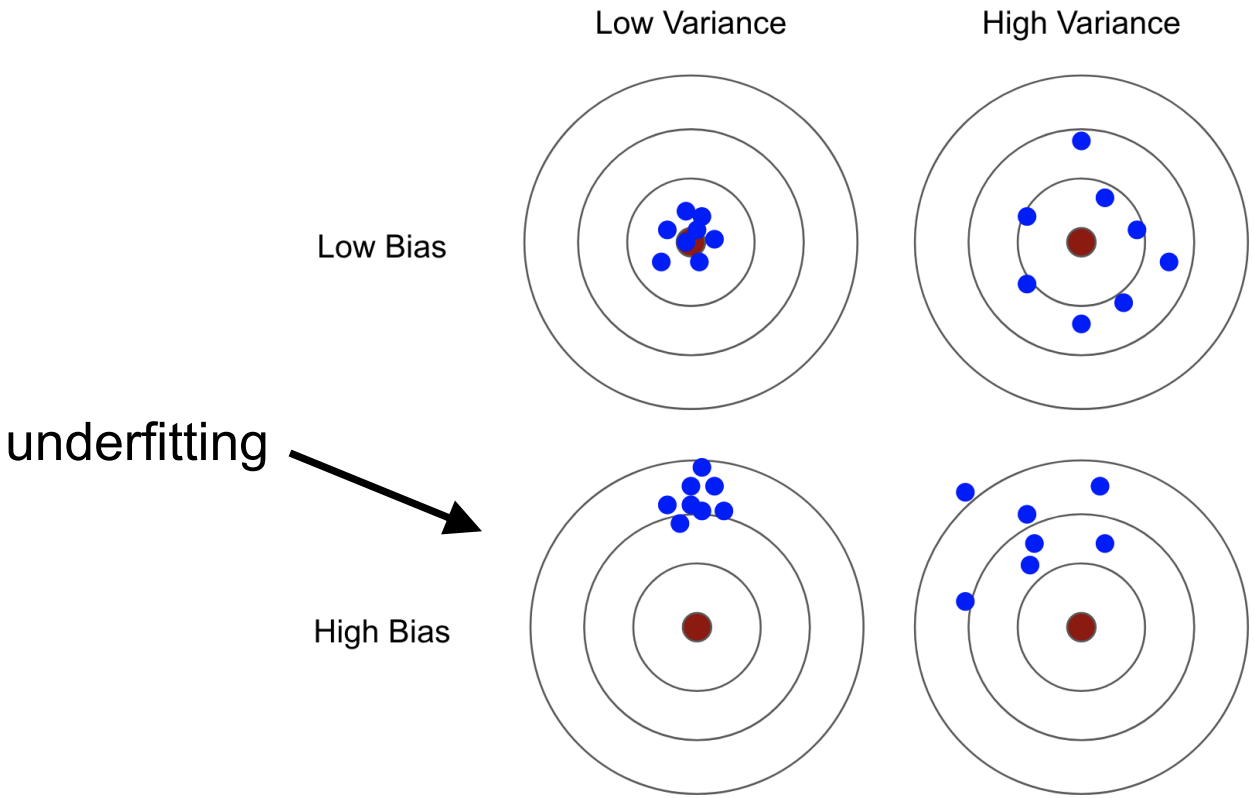




Closer to the center -> better model accuracy

# ML 101: Bias and Variance Tradeoff

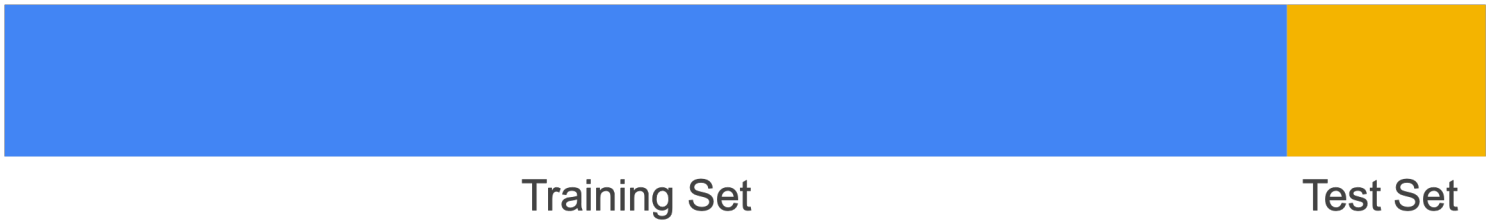
- Use a hold-out test set

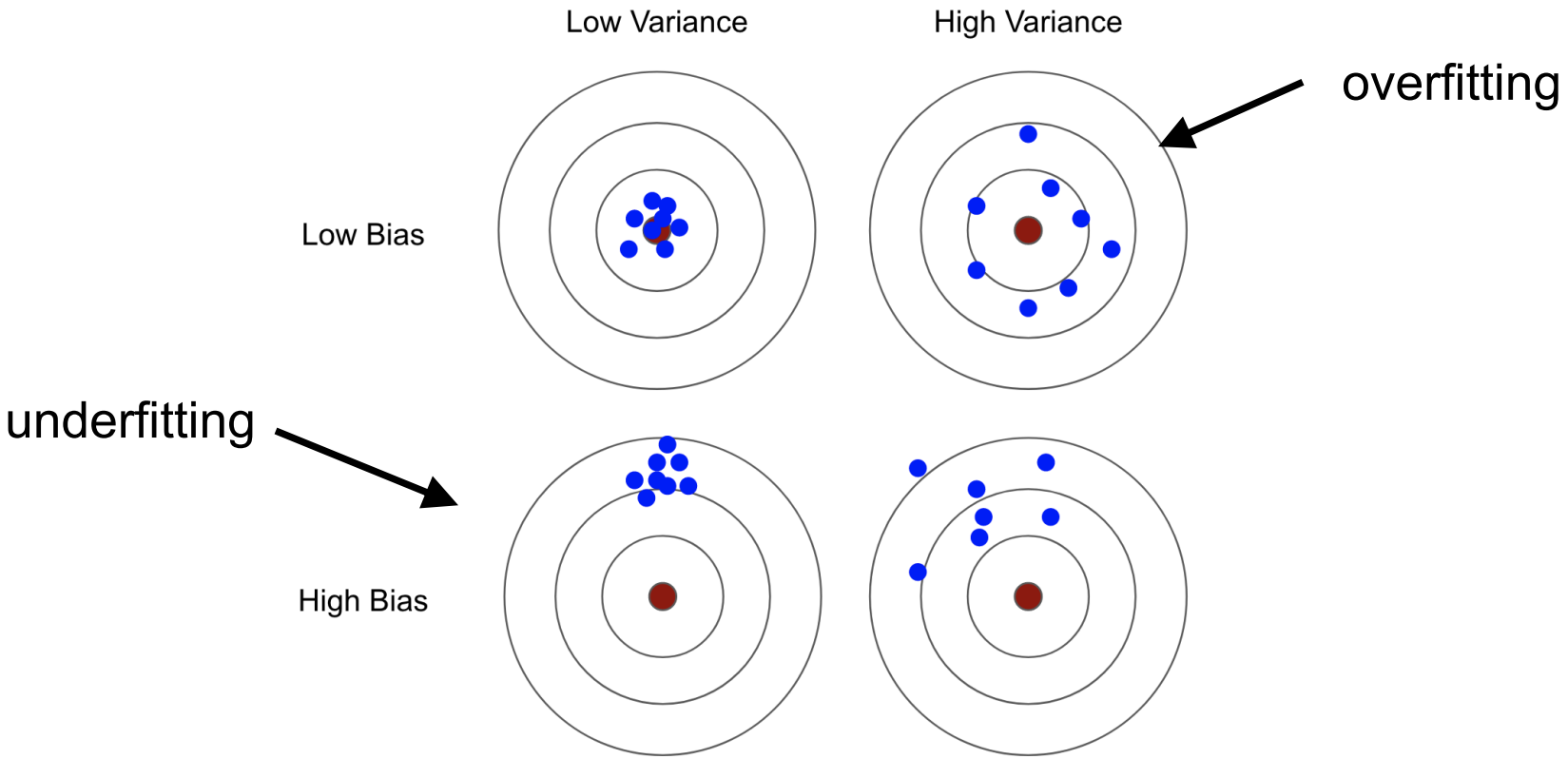


# ML 101: Bias and Variance Tradeoff

- Use a hold-out test set



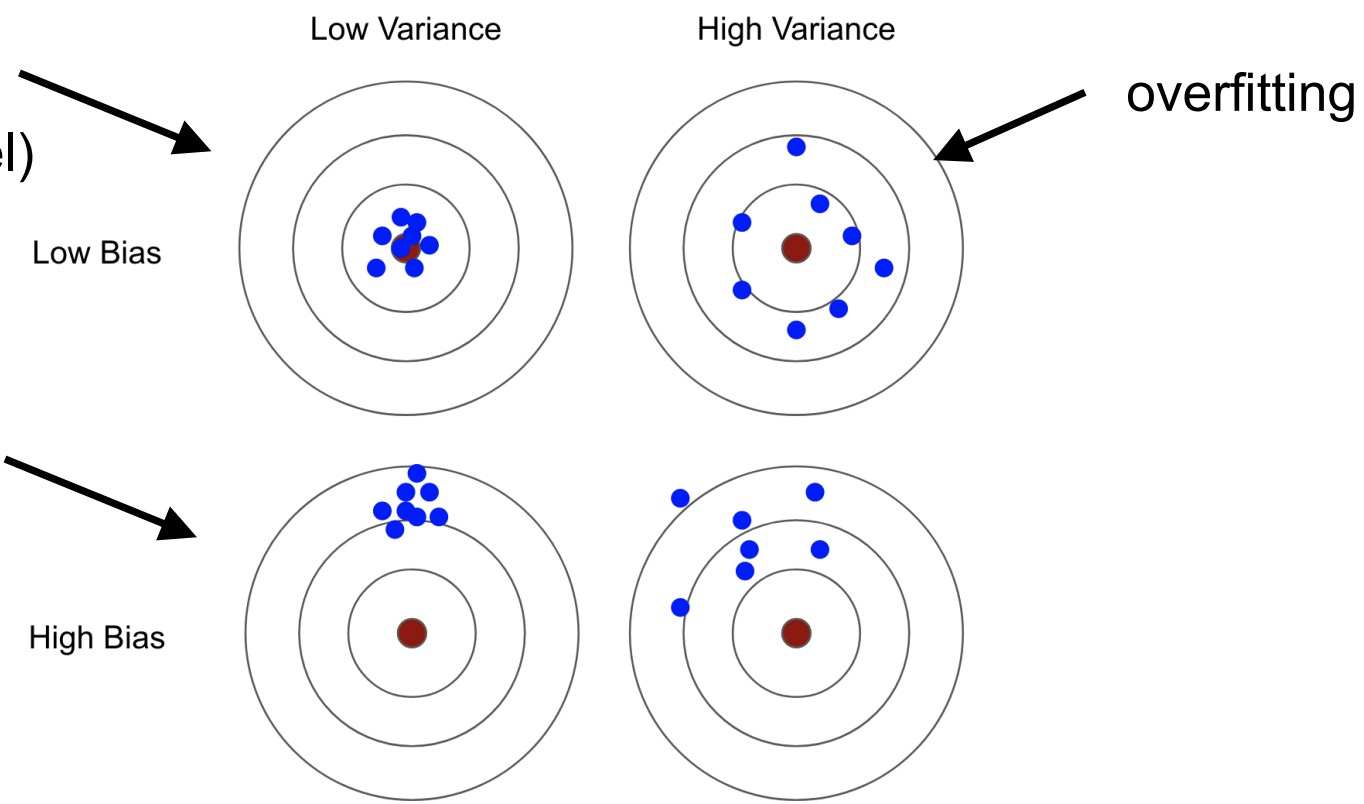



# ML 101: Bias and Variance Tradeoff

- Use a hold-out test set

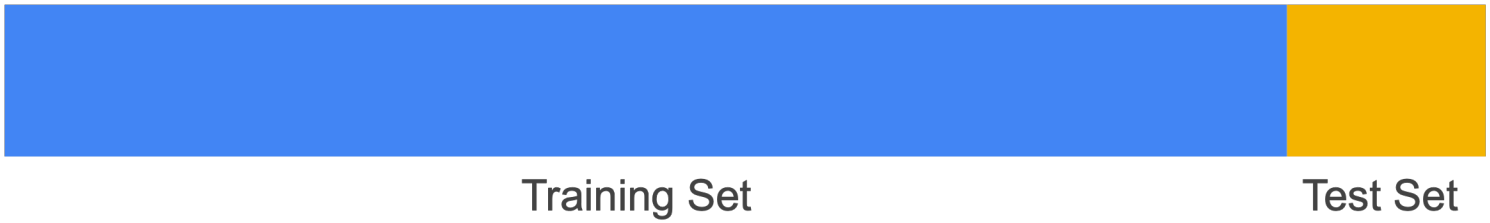


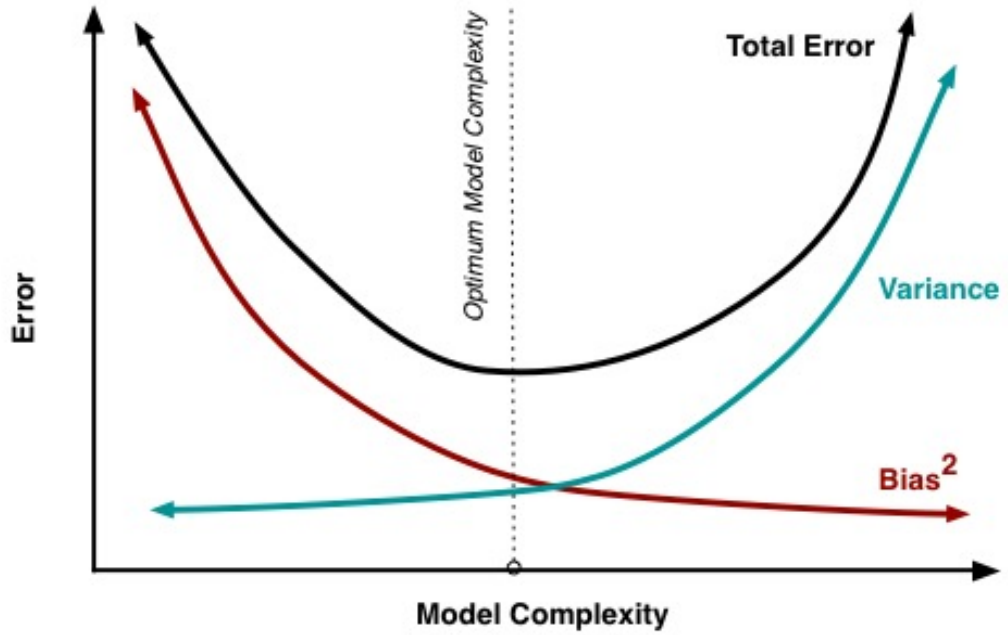

too good to be true? (deep model)



# ML 101: Bias and Variance Tradeoff

- Use a hold-out test set

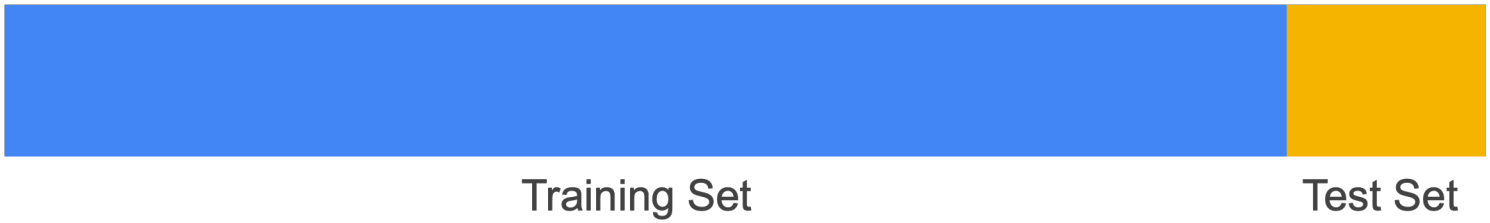



**Deep Models**

# Tradeoff in Deep Models?

- Use a hold-out test set




- Train 100 models, pick the one with best performance on the test set
- Overfitting?
- Not really, if the dataset is sufficiently large and the gain is significant!

See Ben Recht's talk on "**Training on the test set**"
https://www.youtube.com/watch?v=NTz4rJS9BAI